RAG LEARNING DOCUMENTATION

11/08/25

Langchain is a framework for the LLM which is in opensource and which help to optimize the performance and allow to make customization on our own for that purpose we are using Langchain framework

To connect the private info with the LLM we are using the framework called Langchain

It is one of the interference which allow to connect with the different things for our application

Prompt template using for making prompt inbuilt to retrieve we can use prompttemplate

Using langchain we can make all the history to save in memory features which is in-built after this we are going to know about the agents

Agent is used to make our application response in smart as well accurate to the query which is in-built in the langchain

So in overall the features which are all in the langchain is
In-built function :

- Inference
- Memory
- Prompt
- Agent

Langchain framework inbuilt features how used in the RAG is like from the youtube i took insights of like differencing langchain vs langgraph vs langsmith

First about langchain will give these features in RAG:

Start→document retrieval(textloader)->split docs into chunks(charactertextsplitter)-->vector DB(FAISS)--> Retrieval (retrievalQAwith sourceschain)-->LLM→end

1. Langchain is a framework that makes building LLm powered applications easy

   Used mainly to build straight-forward chains and retrieval flows

2. langGraph is a framework to orchestrate multi-step ,stateful workflows using LLMs with a graph-based structure

   Upgrade version of langchain where more advanced compared to langchain

feature	langchain	langGraph
purpose	Toolkit to build LLM apps (chains,tools,agents)	Framework to manage complex workflows with state
style	Linear or reactive chains	Graph-based,supports loops ,retries ,memory
Best use case	Simple chatbots,RAG apps,tool usage	Multi-step workflows ,agents with memory,conditional paths
State handling	Stateless or partial stateful	Fully stateful;remebers and transitions based on logic
Example use	“Book a flight”using a flight API	Plan a vacation(ask budget→choose flights →book hotel→loop if error)

3.langsmith is a debugging and observability platform for LLP apps

12/08/25

Vector database is base for creating dimensions to any form of external data source

Separate into small and small chunks for system understanding

And the dimension number is for the content which is in the any form of external data

For different models there would be different form of dimensions (floating numbers)

Embedding model is need for both the retrieval and for storing it need then only it can able to do the work

Where the user gives the input as query for that also it will change into embeddings

Text→ vector (0.44,0.967,0.234) read and write to match the similar one

Through search it will search the query vector with the stored vector in DB nearest to retrieve the answer to the user query

Text→ token→embeddings both will happen same time

Vector DB structure
{
Id chunk001 value[0.44,0.22,-0.17]
Metadata: source, pageno
}

Tech stack to implement RAG:

- Vector DB:pinecone,chroma DB
- Framework : langchain
- Model embedding: open AI ,groq cloud
- LLM models:chatgpt,claude,gemini

RAG FLOW:

1. User query
2. Retrieves AI search(knowledge base→external source)

Text to vectors store in db
Retrieves the data
3.augmentation (prompt go to LLm for strengthening the answer)
4. Generation (transformers will generate the data)

→Small =384 dimensions (numbers)
→medium=768
→large=1536 upto 3000 there is dimension

If in case using higher dimension = rich information +large storage +slow process
Lower dimension = fast+less storage +limited info

User Question →embedding models→vector→vector DB→RAG process where i mentioned above →LLM transformer →read the question→retrieval docs

Embedding Moddel= Retrieval
LLM transformers = for generation

RAG pipeline

Load source data(Data injection)---------->vector database
Load ,transform,embed

Load -read from specific external source
Transform-feature engineering
Embed - split into chunks
Chunk will convert into vector embeddings which is known as numbers or dimensions

```
from langchain_community.document_loaders import TextLoader
loader=TextLoader("speech.txt")
```

```
text_documents=loader.load()
text_documents
```

Case sensitive
TextLoader accept path as well the filename with the extension
Instead of text depend upon the file it will change the word there
.load() return the list of document object such as page content as well the meta data attribute
Call the text document to show the text which is present in the document

```
 import os
from dotenv import load_dotenv
load_dotenv()

os.environ["OPENAI_API_KEY"] = os.getenv("OPENAI_API_KEY")
```

- To get the open API key we are setting the env to secure the API and after that we are calling and these is the way to call the env of any API keys

```
from langchain_community.document_loaders import WebBaseLoader
import bs4# BeautifulSoup is a library for parsing HTML and XML documents

## load chunk

loader=WebBaseLoader(web_paths=("https://www.youtube.com/watch?v=2Xj8c6a1b0g",),
                     bs_kwargs=dict(parse_only=bs4.SoupStrainer(
class_=("post-title","post-content","post-header")
                      )))

text_documents=loader.load()
```

For any imports for module by using the langchain community will have predefined without need to code anything from the scratch and this is the part of data loading which is the first step

Data loader can be in any form like pdf,docs,txt ,json it can be any form for that loader we need import proper package for working here we have been used webloader because we have attached the weblink as well beautifulsoup used to retrieval from html tags and then

below line is to not for retrieving whole instead what they have mentioned that will be retrieved by using of soupstrainer which is one of the library in the bs4

```
from langchain_community.document_loaders import PyPDFLoader
loader=PyPDFLoader("speech.pdf")
docs=loader.load()
docs

#loader is completed the first step of the RAG pipeline
```

Now it is for pdf we have been imported pyPDFLoader and the same method loading the file and then we can see by calling the assigned name

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
chunk_overlap=200)
documents=text_splitter.split_documents(docs)
documents[:5]

#convert the chunks into vector embeddings
```

And now we step into the second step that is slitting into smaller piece which is called chunks for easy and optimized performance
For that we have been used recursivecharactertextsplitter from same langchain framework which we have seen in the beginning
After that we are giving the limit for chunk size and for the continuation purpose not changing meaning overlap is given and it not goes beyond the chunk size is important
And then the same e have been using split documents and the variable assigned calling the assigned variable to see the answer and for we are going to see first five thats it

```
#chroma DB
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
db=Chroma.from_documents(documents[:20],OpenAIEmbeddings())
```

This part is to store the external source into the database for retrieval for that we have been used chroma in the above code importing and then here we have been used openAI embedding for the docs and it assigned to the db for see the respond or how it saved in the db

```
#FAISS database
from langchain_community.vectorstores import FAISS
db_faiss = FAISS.from_documents(documents[:20], OpenAIEmbeddings())
```

Same format but using different DB FAISS

```
query="who is the author of this article"
result=db.similarity_search(query)
result[0].page_content
```

By using similarity search we are going to retrieve the respond for the query and this can use for both just by changing db into db_faiss thats it

13/08/25

Chunking strategies

- Character splitting

Dont care about the meaning and easy to implement in any languages
Suitable for ocr output

- Recursive charactersplitting

Paragraph→sentences→words→characters
Preserve the context better
Common in langchain recursivecharactertextsplitter
Good for large files
Slightly complex to implement

- Document based chunking

From which document the respond came we need to know then will go for the document based chunking
Instead of splitting the text it will split the document first like of separate unit html,docs,pdf like it will do
Still may break sentences depending on chunking method inside the document

- Semantic chunking

Split is based on meaning not in size
Use embedding or NLP to detect topic gifts and split text when the context changes
Article go from dog breeds to dog training tipe it will detect the shift and split there
High quality and expensive ,slower
All related sentences will be together and also improve retrieval accuracy
q&a bots supporting high

- Agentic chunking

Newer AI-driven chunking method
Uses an LLM as an agent to decide how to split it read the whole text decide logical breakpoints and even annotate chunks with summaries
Human like decision making preserve context
Multi-format awareness(text,tables,code)
Requires LLM calls
Chunk quality is important more that the speed of working like advanced RAG pipelines document analysis,and enterprise search then will go with the agenticc chunking.

Token based chunking:

Splits text based on the model which we are ging to use based on that input limit we need to do the split chunks like GPT-4 handle 8k to 128k tokens ddepending on the version we need to make chunk

Sentence based chunking

Split at sentence boundaries
Preserves readability,but chunks may vary in size which we know
Works well for suummarization tasks

Sliding window chunking:

Overlapping chunks to preserve context between chunks
Great to avoid context loss at split point

Advanced Chunking Techniques

1. **Recursive Chunking**
    - Break large docs → sections → paragraphs → sentences.
    - Useful in hierarchical search.
2. **Adaptive Chunking**
    - Dynamically size chunks depending on content (short paragraphs stay short, long ones split).
3. **Domain-specific Chunking**
    - Legal docs: Chunk by clauses.

- Code: Chunk by functions/classes.
- Academic papers: Chunk by sections.

Embeddings:

Embeddings are vector representations of text(words,sentences or documents)

They capture semantic meaning :similar meanings→similar vectors

Convert both the documents and queries into vector for similarity search

Types of embeddings Used in RAG:

1.word-level embeddings
Represent each word as a vector
Limitation :no content awareness

ex:word2vec,gloVe,fastText

2.contextual Embeedings

Vector depends on the surrounding context of a word or sentence
Generated by transformer-based models

ex:BERT ,RoBERTa

3.Sentence/document embeddings

Represent whole sentence or paragraphs
Optimized for semantic similarity

Popular models:

- OPENAI:text-embedding-ada-002
- HUGGINGFACE:all-MiniLM-L6-v2(via sentence-transformers)-distiluses-base-multilingual-cased
- COHERE:embed-multilingual-v3
- GOOGLE:UniversalSentence Encoder(USE)
- BERT/SBERT:Fine-Tuned BERT for sentence embeddings
- E5:embeddings optimized for retrieval tasks
- InstructorXL:Instruction-tuned embeddings for retrieval
- LaBSE:multilingual embedding from google
- Fastembed:fast &lightweight models from huggingface

1.Open AI text-embedding-ada-002

Highly accurate in semantic search on english
Ideal for large scale ,deployment environment with budget will go with this
Industrial related chatbot this will be suitable

Pros:

Easy to use via API
Small vector size (1536 dims,good performance
Extremely optimized for search and q&a

Cons:
Not multilingual
Paid
Cloud based required internet

2.sentence -BERT all-MiniLM-L6-v2

Small RAG purpose
Local inference own laptop
Offline we can able to use chatbot

Pros:
Open source
Easy to fine-tune
Good accuracy for general queries

Cons:
Not multilingual
Compare to large models quality wise lower
Not specialized in domain specific

3. E5(int/float/e5-base,e5-large-v2)

For searching where we need to do in depth for that it will be suitable
Domain specific document retrieval wll be good here

Pros:
Accurate in retrieval
For searching purpose alone will train this model
Open source runs locally

Cons:
Slightly slower than SBERT
Limited language support
General purpose not that much efficient

4.Instructor Embeddings (hkunlp/instructor-xl)

Where instruction important we can use this for RAG
Instruction following bots

Pros:

Embedding generation is instruction-aware
Good performance for domain specific RAG
Works for both queries and documents

Cons:
Slightly larger and slower
Need careful instruction for best results

5.GTE models

High quality ,open source,alternative to openAI embeddings
Great balance between speed and retrieval quality

Pros

Open source
Good retrieval performance

Cons

Less performance without fine tuning

6.cohere embeddings

Hosted embeddings API with strong multilingual support
Semantic search across multiple languages

Pros

Good coverage
Optimized for search and classification

Cons:

Paid API
Cohere service availability depends

7.jina embeddings

Open source ,high speed embeddings model optimized for retrieval and search

Pros:
Free,open source easy to run locally
Optimized for vector DB use

Cons
Less research backing than big corporate APIs

8.universal sentence encoder(USE)

Universal semantic search

Pros:
Simple to use covers many tasks
Work well for medium sizzed texts

Cons

Outperformed by new models

9.MTEB benchmarks (meta knowledge)

Massive text embedding benchmark current standard leaderboard

14/08/25

Dimensions in embeddings

Represent number of numerical values used to represent a single data item (like documents,paragraph and sentences)in vector space

Each number represents some learned feature about the text -such a meaning,context,relationship to other words etc

Dimension count is fixed for a given embedding model

Dimension length of the vector (number of features)

If the embedding model that outputs 1024 dimensions every vector creates will be length 1024

Higher dimensions usually mean more detailed representation but also morestorage and slower search

Dimensions is important in RAG while retrieving step

Convert document & query to embeddings each vector has a fixed dimensions (eg:384,768,1024)

Store embeddings in vector DB(like pinecone,FAISS,weaviate)

[0.12,-0.45,0.87,....,0.04]--> total 768 numbers example

Affects 👍

Memory usage →higher dimension =more number stored =more storage & RAM usage

Search speed→higher dimensions vectors take longer to compare

Accuracy →Too few dimensions →e,bediings might not capture enough nuance ;

Too many dimensions→can lead to inefficiency

COMMON EMBEDDING DIMENSIONS FOR RAG:

Model	Dimension	Notes
OpenAI text-embedding-3-small	1536	Small,fast,good quality
openAI text -embedding-3-large	3072	Higher accuracy,more memory
Sentence-BERT(all-MiniLM-L6-v2)	384	Lightweight,good for small-scale RAG
mpnet-base-v2	768	Balanced performance
InstructorXL	768	Instruction-tuned

E5-large-v2	1024	Good for semantic search
Cohere embed english-v3.0	1024	Optimized for retrieval tasks
BGE-large-en	1024	High quality for english search

How to choose the dimension

Application goal:

Very high accuracy for complex queries →choose higher-dimension models (eg,1024,1536,3072)

Fast retrieval with low memory usage →choose lower-dimension models(eg,384 or 768)

Speed requirements:

Higher dimensions=more computation=slower similarity search

Lower dimensions=faster but may lose som meaning detail

Storage & cost:

vectorDB size grows with number of vectors * dimension

Model compatibility:
Vector database index mus match exactly the embedding dimension of your chosen model -oherwise wont work

Switching models later means recomputing all embeddings

Chunk size vs dimension

Chunk size how many characters tokens you split yourtext into before embeddings

Dimension how many numbers the embedding model uses to represent any chunk

Chunk size:

Measures in tokens (similar to words ,but includes punctuation and subwords)

Dimension:

Fixed by the embedding model
No matter off chunk tokens the model will still output the same number of dimensions

Input 👍

A chunk of text could be short or long upto the limit its support

Output

Fixed length vector

Its not collapsing text randomly

Number of vectors =how many chunks you've embedded

Dimension how many numbers in each embedding

No of vector * dimension

10,000 chunks and 768 dimension

10,000*768*4=30.7 MB

Index is like a map that helps you to find quickly closest vector without checking every single one

Without an index →its like searching for one specific page in a huge book by reading it cover to cover

With an index→its like using the books table of contents to jump straight to the right page

Makes search fast→especially when we have million of vectors

Saves computing effort and time

Each number in a vector is stored as floating point number like 0.7645

Computers need memory to store each number and 4 byte is a common balance between

- Accuracy
- Storage

Today's overview

1.embedding dimensions

Dimension is how many numbers or features are used to represent each chunk of teit in the vector space

384 dimensional embedding =[0.12,-0.05…]with 384 numbers

Higher dimension can capture more detail /meaning from text ,but cost more storage and processing

2,chunk size vs dimension

Chunk size:how many characters /tokens of text you break your document into before creating embeddings (eg 500-1000)

Dimension :independent from chunk size no matter how big the chunk the embedding will always have the same number of dimension for a given model

Example:

Chunk :”the quich brown fox…”(80 tokesns)
Model text-embedding-3-small(1536 dimensions)
Result A 1536 -length vector for that chunk

3 storage & cost

Storage =no of vectors * dimensions * bytes per number

32 bit float=4 bytes

10,000*1536 dimensions * 4bytes= 61.44. Mb

More dimensions →bigger storage →higher cost to search

4 index in vector database

Index special data structure that organizes vector so you can search them quickly

Without an index →you must compare your query to every vector

With an index→it narrows down to the most likely matches quickly

Think of it like page numbers in a book–without them you have to read the whole book to find a word

5 choosing a dimension

Dimension is fixed by the embedding model you pick not by you

Accuracy

Storage

Speed

6.chunk →model → embedding vector (dimension fixed)-->stored in DB→Indexed for fast search →used in RAg to find relevant context

1.Chunk → Splitting

Small pieces of original text like splitting a book into paragraphs\split a big document into chunks before sending them to the embedding model because models have input limits

Ex:full text “once upon a time in a faraway land …”
Chunk 1: “once upon a time ..”
Chunk 2:”In a faraway land…”

2. Embedding →numerical representation of a chunk created by the model

Computer cant understand text directly embeddings turn words into numbers that preserve meaning

Chunk →[0.12,-0.87,0.33….]a list of numbers

3.vector →another name of embedding list of number

Embedding is what it represents meaning of text vector us the math form

Vector might be 768 numbers long (768=dimension)

4.dimension→how many numbers are in each vector

Bigger dimensions can capture more detail about meaning ,but take more storage and compute

Dimension =3 vector like [0.12,-0.87,0.33]
Dimension =768 →vector like [0.56,-0.12,....768 numbers]

5 feature → each number in the vector is feature - it captures one aspect of the chunks meaning

If dimension =768 → there are 768 features

6 index

A searchable structure in your vector database that stores all vectors so you can quickly find similar ones

[0.78,-0.45,0.12]-->is one vector and also one embedding

Each 3 feature 3 dimension

Chunk = piece of text (or other data) you want to store.

Embedding = vector representation of that chunk.

Vector = list of numbers (features).

Dimension = how many numbers in that vector.

Index = the search-friendly database structure that stores and retrieves your vectors.

Step	Meaning	Example
Chunk	Text piece	`"The cat sat on the"`
Embedding	Model output	`[0.12, -0.34, 0.88, 0.03, 0.45]`
Vector	The list of numbers	same as embedding
Dimension	Vector length	5
Index	Fast search structure	Stored vectors for later retrieval

Embedding is one representation=vector list of numbers that is from embedding=dimension is one number inside that list which is also called feature

17/08/25

Feature	Similarity Search	Semantic Search
Goal	Find items *most similar* to a query vector, mathematically.	Find items *most relevant* to the meaning of the query.
Basis	Purely distance-based (cosine similarity, Euclidean distance, dot product).	Distance-based **+ semantic meaning from embeddings**.
Example	Query: “dog” → returns “puppy”, “hound”, “wolf” if embeddings are close.	Query: “dog” → returns “pet animal”, “canine companions” even if words differ.
How	Works with any numeric vectors (images, audio, text).	Requires embeddings trained to capture **meaning**, not just surface features.

In RAG	Retrieves chunks that are closest in vector space.	Retrieves chunks that feel *conceptually related* even with different wording.

Index

Yes — every database that supports vector search has **some form of index**.
Without an index, the database would need to compare your query vector against **every single stored vector**.

- Index = that helps quickly find “nearest neighbors” without scanning the entire DB.
- In vector DBs, common indexing methods include:
    - **HNSW** (Hierarchical Navigable Small World graph) → used in Pinecone, Qdrant, Weaviate.
    - **IVF + PQ** (Inverted File + Product Quantization) → common in FAISS.
    - **Annoy** → tree-based approach.
- Index building happens **once when you insert/update** vectors, so future searches are faster.

18/08/25

Vector Database:stores the data for retrieval process

Using keyword search by the method of normal database it couldnt able to retrieve the context or sentence which matches so we have gone through the vector database

Where it stored embeddings (list of numbers that capture meaning)for each chunk of text

So, in RAG:

1. **Documents** → split into chunks → embeddings → stored in a vector DB.
2. **Query** → embedded → DB finds nearest chunks using similarity search (cosine/dot/Euclidean).
3. Chunks are passed into LLM → final answer.

Common in all DataBases

Vectors & Embeddings – *Numbers that represent meaning.*

- Example: “cat” → `[0.12, -0.98, 0.44]`, “dog” → `[0.15, -1.01, 0.40]` (close vectors = similar meaning).

Indexes – *Structures that speed up search.*

- Example: Library catalog helps find “Harry Potter” fast instead of checking every book one by one.

Distance Metrics – *Math to measure closeness.*

- Example: Cosine similarity → “cat” is closer to “dog” than to “car.”

Similarity Search (kNN) – *Find nearest neighbors to a query.*

- Example: Search “AI” → DB returns 5 most similar stored docs (e.g., “machine learning,” “neural networks”).

Metadata Filtering – *Extra info to refine results.*

- Example: Search “AI articles” → filter to only those published in 2023.

Hybrid Search – *Mix of keyword + vector search.*

- Example: Search “iPhone 13 battery life” → results must match both the exact keyword “iPhone 13” AND semantically similar docs.

Scalability & Sharding – *Split data across machines for big scale.*

- Example: Netflix stores billions of movies/users; vector DB shards data so no single server is overloaded.

types:

Pure Vector Databases (built only for vector search)

- Pinecone, Milvus, Qdrant, Weaviate.
- Optimized for embeddings, indexing, distributed scale.
- Best when RAG is the core use case.

Vector Extensions on Traditional Databases

- PostgreSQL + pgvector, MongoDB Atlas Vector Search, Elasticsearch with dense_vector.
- Add vector search on top of existing DB.
- Good if you already use SQL/NoSQL and want RAG without moving data.

Embedded / Local Vector DBs (lightweight, runs inside apps)

- Chroma, LanceDB, SQLite + Vec.
- Easy to use for prototyping, small projects, personal RAG apps.
- No big infra needed.

One by one explanation

- **Pinecone** (hosted, SaaS, production-scale)
- **Weaviate** (open-source, hybrid semantic + symbolic search)
- **Qdrant** (open-source, Rust-based, fast + simple API)
- **Milvus** (one of the earliest open-source vector DBs, from Zilliz)
- **Chroma** (lightweight, popular for RAG prototypes, used in LangChain by default)
- **LanceDB** (new, parquet-like storage + vector search)
- **pgvector (Postgres extension)**
- **Elasticsearch / Opensearch** (hybrid keyword + vector search, used in enterprises)

Pinecone - First Saas vector database

Before pinecone (2020),if we want vector search then we have to write our own ANN(Approximate nearest Neighbour)algorithms like HNSw,FAISS or annoy

Manage indexing,sharding(splitting the data into pieces spread across many machines when the user ask query the database asks all shards and combine the result),scaling and distributed systems

Pinecone was built as fully managed,cloud-based vector database

Send vectors and queries through an API
No need to do anything extra how we do in the FAISS or ANN

Metadata is manual we need to give or else it goes by default with ID

ID is also manual given by developer to find the particular thing -NOT AUTO GENERATE

Working strategy :

Text→break into chunks

Generate embeddings with openAI,huggingface ,cohere embeddings

Store embeddings +metadata in pinecone

Each record={id,vector,metadata }

Query comes in→embedding generated

Pinecone finds nearst vectors using similarity search (cosine,dot product ,euclidean)

Return top k-documents -> passed to LLM

Pros:

Serverless scaling→ no need to manage clusters

Index types:

HNSW hierarchical Navigable small world → fast ANN
IVF (Inverted File) for very large datasets
Hybrid search (vector + filter)

Filter : year >= 2020 (metadata filter - when question answer? push vector +metadata together later using filter. eg search vector but only year >=2020)

Namespaces →like folders for data ,useful for multi-tenant apps (folders or partition inside database useful if you want to keep the data separate but in same DB queries only happen inside the right folder so other wont able to see this data)

Persistence →data stays safe ,unlike in-memory FAISS

No infra headache →managed service(interact only with API no setup no worrying about crashes)

Great docs & support used in many RAG tutorials

Cons:

Closed source can't self host (run the database on our own server or cloud ,hard to manage but own it fully)

Paid Saas free tier is limited ,big projects get costly

Pinecone cloud dependent less control compared to open source like quadrant or weaviate

Only one index type is supported since pinecone is fully managed it abstract away low-level choices

1. Create an index

- Choose a **dimension** = size of embedding model (e.g. `1536` for OpenAI embeddings).
- Example:

```
pinecone.create_index("my-index", dimension=1536, metric="cosine")
```

Metric = how similarity is measured

When the pinecone compares query vector with stored vector it needs a math rule

1.Cosine similarity →measures the angle between two vectors

-> good for text embeddings (direct matters,not magnitude)
->range -1 to 1 (closer to 1 =more similar)

2.Dot product →measures overlap (like projection)

Sometimes used in recommendation systems

3.Euclidean distance(L2)-->straight line distance between vectors

More commonly in image/vector spaces than text

LLM+embeddings (openA,cohere and many more)cosine is the best default

If the result were:

- **1** → exactly the same direction (perfectly similar).
- **0** → 90° apart (no similarity).
- **-1** → opposite directions (completely different).

1. Similarity Search (the “math” layer)

This is where **cosine / dot product / Euclidean** come in.
They are just *formulas* that tell us **how close two vectors are**.

- Think of them as rulers with slightly different measurement styles.

Semantic Search (the “meaning” layer)

This is about **what the vectors actually represent**.

- We first take text (or images, or audio) → convert them into **embeddings** using a model (like OpenAI embeddings).
- Those embeddings capture **semantic meaning** (e.g., “king” and “queen” are close, “cat” and “dog” are closer than “cat” and “car”).

Then the **similarity search formulas** (cosine, dot product, etc.) are applied on those embeddings to **find semantically similar things**.

So basically:

- **Similarity search = the math operation** (how close are vectors?)
- **Semantic search = the application** (using embeddings + similarity search to find meaning-based matches)

Example:

- You ask a RAG chatbot: *“What is the capital of France?”*
- Your query → converted into an embedding (vector).
- Database contains embeddings of documents.
- The vector DB uses **cosine similarity (or dot product, or Euclidean)** to find the **most semantically similar chunks**.

---

2. Upsert vectors (store docs)

- Each vector needs an **ID** (string, you choose).
- Optionally add **metadata** (e.g., `{"source": "pdf1", "page": 2}`).

```
index.upsert([
    ("doc1", embedding_vector1, {"source": "pdf1", "page": 1}),
    ("doc2", embedding_vector2, {"source": "pdf1", "page": 2})
])
```

---

3. Query for similarity

- Turn user query into an embedding.
- Search Pinecone for nearest vectors.

```
result = index.query(vector=query_embedding, top_k=3,
include_metadata=True)
```

---

4. Retrieve documents

- Extract the matched texts/metadata.
- Example: get top 3 most relevant docs for the query.

---

5. Generate answer

- Pass both the **query** + **retrieved docs** into your LLM.
- LLM synthesizes a final response.

Key Notes

- **ID** → always required, you choose it (manual, usually `uuid` or doc name).
- **Namespace** → optional, use if you want multiple separate collections inside same index (like "pdfs" vs "faqs").
- **Metadata** → optional, but super useful for filtering (like “only return docs from 2023”).
- **Indexing** → automatic (Pinecone manages the internals).

Search types in vector database

1.Similarity search (core one)-->find vector closest to your query vector

Vector search will go with this most common

2.Hybrid search

Combines vector search (semantic)+ keyword search (lexical)

Useful when you want both exact word matches and semantic meaning

Query: *“Apple latest”*
Vector Match: “Apple iPhone 15 launch”
Keyword Match: “Apple fruit harvest 2023”
Final Results: Both show up, because one is **semantic match** (Apple = company) and one is **keyword match** (Apple = fruit).

3.Metadata filtering

Search by similarity plus filter

Date range

author=john

Category =science

Ex:find similar documents ,but only from 2023

- **Query:** *“climate change solutions”*
- **Filter:** `year = 2023`
- **DB Returns:**
    - “2023 UN report on renewable energy”
    - “2023 research on carbon capture”

(Older docs like from 2018 won’t show, even if similar.)

4.full-text /keyword search (outside vector space)

Standard keyword search alongside embeddings

- **Query:** *“machine learning”*
- **DB Returns:**
    - Only docs that literally contain the phrase *“machine learning”*
      (No semantic flexibility: if the doc says “AI training,” it won’t match.)

Milvus DB-First open source

Open source not fully managed as like pinecone but you can host it yourself

Similarity search and AI apps

2019 found by zilliz ,linux foundation project (LF AI & DATA)

Runs anywhere :laptop ,kubernetes or managed cloud

Scalable-handle billion of vectors

Index can create by own or choose algorithm which mention above

Manually assign ID if want control or auto -generate by default behaviour

Namespace (folders)-Uses SQL Collection

Support multiple indexes types (more than pinecone)

- IVF(inverted file index)-before searching it will group the students same group the vectors then it will search
- HNSW(hirearchial navigable small world graph)-instead of checking every vector by implementing shortcuts
- Annoy (approximate nearest neighbour)-approximate hints instead of exact matches
- Flat (brute-force,exact)-every vector check
- IVF_SQ8
- IVF_PQ

Integrate with GPU's →faster searching

Strong community (popular for Ai +enterprise use cases)

Support hybrid search wih vector similarity

Multiple indexes support

Active community

Integration-friendly → works with langchain,huggingface,pytorch

Applications:

AI-powered search
Recommendation system
RAG chatbots
video/audio retrieval
Enterprise AI apps that need scalability +self-hosting

Cons

Setup is complex compared to pinecone

Managing service by our own scaling ,sharding ,monitoring ,upgrades

Overkill for small projects

Query tuning needed →pick indexes types and parameters ourself

Index Type	How it Works (1-line meaning)	Pros	Cons	Best Use Case
FLAT (a.k.a brute force)	Compares **every vector** one by one.	100% accurate (exact search) Simple to use	Very slow for large datasets High memory cost	Small datasets where accuracy is critical
IVF_FLAT (Inverted File)	Groups vectors into **clusters (centroids)** first, then searches only inside closest clusters.	Much faster than FLAT on large data Scales well	May miss some neighbors (approximate) Needs tuning (#clusters)	Large datasets with balance between speed & accuracy
IVF_SQ8 (IVF + Scalar Quantization)	Same as IVF, but vectors are **compressed to 8-bit integers** inside clusters.	Saves a lot of memory Faster query speed	Lower accuracy due to quantization Extra training step	When memory is limited but dataset is huge
IVF_PQ (IVF + Product Quantization)	Same as IVF, but splits vectors into parts & compresses each part.	Huge memory savings Handles billion-scale datasets	Accuracy drops compared to IVF_FLAT More complex to tune	Ultra-large datasets (billion+ vectors)
HNSW (Hierarchical Navigable Small World graph)	Builds a **graph of vectors with shortcut links**, so search follows paths instead of scanning all.	Very fast & accurate Great for high-dimensional vectors	High memory usage Slower index build time	Real-time search where speed matters most
Annoy (Approximate Nearest Neighbor)	Builds **random projection trees** and searches approximate paths.	Lightweight Good for read-heavy workloads	Less accurate than HNSW Slower updates	Recommendation system,offline workloads

Chroma DB-open source,light weight and dev-friendly

Built mostly for AI/LLM apps

Lightweight ,easy to use ,runs inside your app

Key features

Schema-lite -you dont need heavy setup (just create a collection and insert)

Persistent or in -memory storage

Simple API

Handles embeddings+metadata together

Indexing

Default-mostly flat(checks every vector)

Good for smaller / mid scale projects

For huge scale ANN→milvus /weaviate

Cons

Not readymade cloud service like pinecone does if want we can setup

Not scalable

Limited index types

Not good for enterprises

Work for more than <1M embeddings,but slow down beyond that

No multiple indexes

Persistent storage(saving data permanently) :simple (file-base DB) SQlite using store the data in files

Can plug into postgres for more reliability

Support in-memory only (not saved restart again it gone)(fast ,but not persistent)

Popular in the langchain ecosystem

Works smoothly in openAI,huggingface,cohere embeddings

Fits directly into chatbots /QA bots pipelines

Run on my laptop

Server mode(running as a service) for multiple clients

Without server :it can only use by me and see by me
With server :chroma start on port 8000 -your chatbot app and search app and friend program can all connect to the same chroma database

Not cloud managed like pinecone

Some try to **auto-create defaults** (Chroma, Qdrant).

It just **hardwires FAISS inside** and builds a *default index automatically*.

Open-source DBs (Milvus, Chroma, Weaviate) → you usually must choose/configure indexes.

Fully-managed DBs (Pinecone, Qdrant Cloud, Vespa Cloud) → indexes are auto-managed.

Feature / DB	Pinecone	Milvus	Chroma	Weaviate	Qdrant	Vespa
Open Source?	No (managed service only)	Yes (Apache 2.0)	Yes	Yes	Yes	Yes
Deployment	Cloud only (fully managed)	Cloud or self-host	Local or server	Cloud + self-host	Cloud + self-host	Self-host or cloud
Indexing	Auto (abstracted, hidden from user)	Many options (IVF, HNSW, etc.)	Auto (FAISS under hood)	Auto + some customization	HNSW (default, auto)	HNSW + advanced search

Persistence (storage)	Cloud backend (abstracted)	RocksDB, MySQL, etc.	SQLite + Parquet (simple local files)	Built-in	Built-in (storage engine)	Own engine, full persistence
Scalability	Very high (multi-region, production-grade)	Very high (billions of vectors, distributed)	Low–medium (good for prototyping & small apps)	High (supports scaling, hybrid search)	Medium–high (clusters supported)	Very high (enterprise scale)
Ease of Use	Easiest (no setup)	Medium (more configs)	Very easy (dev-friendly, plug & play)	Medium (some learning curve)	Easy (simple API, Rust core)	Harder (complex but powerful)
Server + Clients	Always server mode (managed API)	Run as server, multiple clients connect	Optional server mode (otherwise local-only)	Server + multiple clients	Server + multiple clients	Server only (enterprise style)
Unique Features	Reliability, production-ready SLAs	Flexible indexing, plugin ecosystem	Simplicity, quick prototyping	Hybrid search (text + vector), GraphQL API	Strong Rust backend, reliability, filters	Handles search + recommendation at enterprise scale
Best Use Cases	Enterprise apps, massive production workloads	Any scale ML/RAG apps, research + production	Quick RAG prototypes, local dev, small projects	Semantic search + hybrid ML apps	Fast + reliable vector DB for mid-scale apps	Complex enterprise search & recommendations
Limitations	Vendor lock-in, not open	More setup overhead	Not scalable for billions	Learning curve, some overhead	Not as battle-tested at very	Steep learning curve, heavier infra

huge
scale

- **Pinecone** = best for **cloud-only production**.
- **Milvus** = best for **flexible, scalable, open-source** workloads.
- **Chroma** = best for **local dev + small RAG apps**.
- **Weaviate** = best for **hybrid semantic + vector search**.
- **Qdrant** = best for **mid-scale, fast, reliable open-source**.
- **Vespa** = best for **big enterprise, advanced recommendation/search**.

RETRIEVAL PART

1.Naive or Similarity search :find most similar chunks (piece of text) to your query using vector distance (like cosine similarity) closest in meaning to the query

If need closest match without any filtering then it would be a best choice

2.Maximal Marginal Relevance : balance between relevancy close match and diversity different perspective prevents duplicates

“History of AI”

For this query it will think different perspectives as well the relevant meaning

3.hybrid search combines keyword search(BM25) exact word match with vector search semantic meaning

When need for both meaning and exact word matches
Complex infrastructure
Slower than pure vector search

What is Newton’s law?”

For this query it will give the exact word as well the meaning or relevant answer

4.filtering retrieval based on the rules or conditions it will retrieve the answers with the semantic search

Query = “latest Apple earnings report”
Retriever does: `similarity_search(query, filter={"year": 2024})`

5.cross encoder which is called re-ranking firstly take retrieval roughly and then re-check with the strongest model like BERT

System-level reranking = reranking done automatically by your RAG framework.

Re-ranking retrieval strategy = you explicitly add a reranker model.

Quality is important more than speed then we can go for this

Query benefits of solar energy first it retrieve 10 documents and then inside that top 3 it will retrieve

Need GPU and slower

6.Multi-query expansion-turn one query into multiple variations to cover different angles When the query is vague,ambiguous ,short

Apple is a query then it will go for apple is a iphone and then apple is a fruit like in variate=ions it will retrieve

More computation and overthink

7.knowledge graph retrieval instead of just matching text it will build a graph of entities (people ,places,cocepts) and search connections

If we want relationship who working for whom cause and effect connection for these type of answers will go for this who is founded openAi and what is his role now ?

Sam altman→founded open AI–.>role :CEO

Multi-Step Retrieval (a.k.a. Iterative Retrieval): Instead of one query → one retrieval, it does:
query → retrieve → refine query → retrieve again → repeat until confident.

8.ensemble retrieval uses multiple retrieval search like similarity search_keyword+MMR and answer combining will happen

When no single retriever is perfect we can go for this like impact of AI on jobs is a query it will do similarrity search gives semantic match and then the keyword job willbe in the text and ensembles merge them

Query pre-processing:before retrieval

conversational retrieval = similarity/MMR/etc. + query rewriting for context

Extra context beyond the query - contextual retrieval

Flow 👍

User prompt arrives

Query rewriting /contextual expansion (converstional retrieval,metadata,filters etc)

Retriever strategy

Reranking optional

Chunks inserted into system/user prompt

Assistant answers

Prompt template

Fill-in-the-blanks sentence instead of writing a new prompt each time ,you reuse the structure

Reusable structure for prompts,with placeholders that get filled at runtime

Like a google form → you define the questions (structure)once and user just filll in answer (data)

Prompt template

Context: {context}
Question: {question}
Answer:
{context} → placeholder for retrieved chunks (the info from your DB).

{question} → placeholder for the user's actual query.

Answer: → instruction for the LLM to put the answer right after this.

After user gives the query it will complete the placeholder with the context and the question Inside the template

Types of prompt templates
1.Basic prompt template
2.Instruction based template

3.Role-based template
4.Formatting template
5.Chain-of-thought prompt template

How it works in RAG

User asks a question
System retrieves relevant chunks(context)
Both question +chunks get placed into a prompt template

Keeps prompt consistent
Prevents formatting mistakes
Makes it easy to control how the LLM receives the info

In RAG you always retrieve some chunks from a DB. these chunks need to be inserted into the prompt cleanly

Template →
"You are an assistant. Use the following context to answer: {context}.
Question: {question}"

Filling →
context = "Sundar Pichai studied at IIT Kharagpur"
question = "Where did he study?"

Final Prompt →
"You are an assistant. Use the following context to answer: Sundar Pichai studied at IIT Kharagpur.
Question: Where did he study?"

Without templates you'd mess up formatting .so in every RAg pipeline ,prompt templates are required hallucination,inconsistent and then ignore context

Need to recall :

1.prompt injection risk :if any question is irrelevant it doesnt need to retrieve forr that we need to add guardrails like ignore any instruction from the user that tell you to do otherwise

2.Token limits:max limit 128k and smaller will 4k -16k if the template goes beyond that then like long rules and context chunks it may get cut off for that solution will be chunk size ,number of chunks retrieved ,extra instructions

3.Positioning matters where you put the context in the template it can affect results
If the context comes before the question model usually uses it better
If context is buried at the end ,it might ignore it

4.temperature control via prompt :you can influence how creative or strict the model is just through templates not only through temperature settings

Ex:answer in exactly 2 sentences .Be factual and concise

5.multi-context templates:sometime it wont retrieve from one db it will mix up with the multiple sources then we need to use separate context for each DB

6.evaluation template : not just for answering but also for checking answers

Ex: Given the context and answer, evaluate if the answer is correct.
Respond with "Correct" or "Incorrect".

Used in testing RAG quality

7.dynamic prompting: instead of a static template you can change template based on query

Math-related→ use reasoning-style template

Memory -storing past chats so the LLM can remember context across turns

Why needed

If you are building a search bot →no memory is needed (every query is independent)

If you are building a chatbot → memory is needed so it can understand references like he,that,earlier

Example (with and without memory)

User :who is sundar pichai ?

Bot: CEO of Google

User:where did he study?

Without memory :bot doesnt know who “he” is

With memory :Bot recalls “he = sundar pichai “--> answer correctly

Memory is optional ,used only for conversational RAG

Types of memory in langchain

Short -term buffer memory —>last past small chats
Long-term memory→stores important factor for later recall
conversationBufferMemory→saves full conversation
Conversationsummarymemory→saves short summary
Vectorstorememory→store embeddings of past chats for small recall

When to use

Needed if chatbot =conversational
Not needed if app=one shot Q&A

Agents (advanced/optional)

Agent = an LLM that doesnt just answer but also decides what to do next (which tool or step to call)

LLM+decision maker

Decide what to do next ,not just answer

RAG gives knowledge;agent gives actions

Agent do?

Break a complex query into steps
Calls tools (DB,calculator,search engine,API)
Decide when to stop and give the real answer

Types of agent

1.ReAct(reason+act)

Think step→act→observe result→think→act again

“What’s the weather in the city where Sundar Pichai was born?”

- Step 1 (Reason): Sundar Pichai was born in *Chennai*.
- Step 2 (Act): Call weather API for *Chennai*.
- Step 3 (Answer): “It’s 30°C in Chennai today.”

ReAct agents are like a detective who gathers clues step by step

2.Tool using agents :designed with specific tools

User: “What is Sundar Pichai’s age squared?”

- Agent → gets birth year from Wikipedia (1972).

- Agent → calculates age (2025 − 1972 = 53).
- Agent → does $53^2 = 2809$.
- Answer: “Sundar Pichai’s age squared is 2809.”

3.Planner +executor:
planner agent →makes a plan
Executor agent →runs it

User: “Summarize Sundar Pichai’s career and make a timeline chart.”

- Planner Agent → Plan: (1) Collect data from DB, (2) Summarize, (3) Call chart tool.
- Executor Agent → Runs each step.
- Final Answer: A neat timeline chart.

4.conversational agent →past chats will be remembered
5.specialized domain agent →tailored for one filed

Need of Agents in RAG

For advanced tasks → need agents.

- Example: do math, summarize + search extra info, or call external APIs.

Assigning tasks to tools

User query comes in
Agent analyzes →what steps are needed ?
Agent calls tools in the right order
Combine results→final answer

Regular RAG =retrieve + generate
Agentic RAG =retrieve +choose best tools + generate
Needed when queries are multi-step or require external actions

LLM concept in RAG

Flow

query → tokens → embeddings → DB → chunks → prompt → LLM → decoding → final answer)

Tokenization = text → small pieces (tokens).

Embedding = tokens → vector of numbers.

Index = map for fast search.

Chunks = grouped text stored in DB.

Retrieval = find top-k relevant chunks.

Prompt Template = combine rules + query + chunks.

Inference = model “thinks” using attention.

Decoding = how next word is chosen.

Post-processing = cleanup, safety, formatting.

Final Answer = grounded response to user.

Features and tricks LLM in RAG

1. **Grounding** = force model to use context, not imagination.
2. **Hallucination control** = use re-ranking, critics, or verification.
3. **Fine-tuning/Adapters** = adjust LLM behavior for domain.
4. **Context window** = limit of tokens model can see (e.g., 4k, 8k, 32k).memory span or how much text it can read in one go like gpt 3.5 will read 4000 tokens and gpt4 turbo will read 128000
5. **Latency & Cost controls** = fewer chunks, caching, compression.
6. **System/User/Assistant roles** = structured prompt format.

Step 1. User Query

- Input: “Why is my washing machine noisy?”
- This is just plain text.

Step 2. Tokenization

Break the text into **tokens** (small units).

- **Token** = smallest unit LLM understands (not always full words).
- Techniques:
    - **Word-level** (each word = 1 token)
    - **Character-level** (each letter = 1 token)
    - **Subword/BPE (Byte Pair Encoding)** → common method in GPT (splits words into frequent pieces, e.g., “wash”, “ing”).
    - **Unigram (SentencePiece, LLaMA)** → probabilistic way of splitting words.

Example:
“Washing machine noisy” → `[“Wash”, “ing”, “ machine”, “ nois”, “y”]` (5 tokens)

Step 3. Embedding Creation

Convert query tokens into a **vector** (list of numbers).

- **Embedding** = numeric representation of text meaning.
- Example: “Why noisy?” → `[0.12, -0.83, 0.45, …]` (768-dim vector).

Step 4. Retrieval from Database

Use embeddings to **search DB** for similar chunks.

- **Vector DB** compares query vector vs stored vectors.
- **Index** = a structure that makes search fast (like a map).
- **Metadata** = extra tags (source, date, doc ID).

Example:
From manual, DB retrieves chunks like:

- Chunk 1: “Spin cycle noise can be caused by unbalanced loads.”
- Chunk 2: “If bearings are worn, replacement is needed.”

Step 5. Select Top Chunks

Choose the most relevant chunks (usually Top-k = 3–6).

- **Top-k retrieval** = pick k most similar pieces.
- Tradeoff: higher k = more context, but more cost & risk of confusion.

Step 6. Build Prompt Template

Combine **system rules + retrieved chunks + user query**.

System Prompt (rules) = hidden instructions (role of model).
User Prompt (query) = actual question.
Context (retrieved chunks) = knowledge we want to ground answers in.

🧩 Example:

```
System: You are a washing machine troubleshooting assistant.
Use only the provided context to answer.
If unsure, say "I don't know."

Context:
1. Spin cycle noise can be caused by unbalanced loads.
2. If bearings are worn, replacement is needed.

Question: Why is my washing machine noisy?
Answer:
```

Step 7. Inference (Forward Pass)

The LLM reads the full prompt.

Inside:

- **Transformer layers** → neural network that processes tokens.
- **Self-attention** = each token "looks" at others in the question (e.g., "noise" ↔ "spin cycle").
- **Cross-attention** = model also looks at context chunks.

Step 8. Decoding (Answer Generation)

How the model decides next word.

Techniques:

- **Greedy** → pick most likely token each step (strict).
- **Top-k** → randomly pick among k most likely (variety).
- **Top-p (nucleus)** → pick from smallest set whose probability ≥ p.
- **Temperature** → controls randomness (low = strict, high = creative).
- **Beam search** → explore multiple possible sentences (rare for QA).
- **Constrained decoding** → force output format (e.g., JSON or citations).

🧩 Example (low temp, greedy):
Answer: “Your washing machine may be noisy due to unbalanced loads or worn bearings.”

Step 9. Post-Processing

Clean and check the output.

- **Detokenization** = turn tokens → text.
- **Stop sequences** = cut output at certain marker.
- **Schema enforcement** = force JSON/Markdown style.
- **Citation mapping** = ensure [1], [2] map to chunks.
- **Safety passes** = remove PII (personal info), profanity.
- **Verification** = 2nd model checks if answer is grounded.

Step 10. Final Answer

User gets a clear, safe, grounded response:

“Your washing machine may be noisy due to unbalanced loads or worn bearings.”

QUERIES

Tokenization 4types

Subword /BPE break rare words into smaller parts but keep common onew whole ex washing →[wash ing]
Unigram probabilistic method (model tries many splits,picks best one learned from training)
Washing migh be split [‘wash’ ‘ing’]or[‘wa’, ‘shing’] depending on context

Default vs choosing you dont pick manually each LLM comes with a built in tokenizer

GPT uses BPE
LLama uses Unigram
Old ones may use word /character

Use GPT or cohere ,tokenizatio is automatic

Chunking

Top-k retrieval –always pick best k chunks
Dynamic k – pick as many as needed until similarity score drops
Re-ranking get top -20 then reorder by better scoring model
Hybrid retrieval =mixed keyword +vector search

How to choose?

For QA bots→ top 3 to top 5 chunks
For summaries →higher k (10-20)
For sensitive apps→use re-ranking for accuracy

In RAG → **fixed-length + overlap** is most popular (ex: 500 tokens, 50 overlap).
Semantic chunking is better for accuracy, but more complex to implement.

Inference

Thinking step inside the model

Technical :forward pass through transformers layers

Model looks all tokens + context together and build an understanding before predicting next word

Running the trained model with your input to predict the next token.

- Input tokens = “Washing machine is making”
- Model predicts next token → “noise” (highest probability).

- Then continues step by step: “due”, “to”, “bearings.”

Inference = turning input into probabilities for the next token

Attention Mechanism : the process which model decides which tokens matter most when generating the next word
Like highlighting important words in a sentence

Self attention-token in the input attend to each other
Ex: the dog chased the ball →model sees “dog”is linked to “chased”
cross attention-token in the question attend to tokens in retrieved chunks
Ex: user asks why is my washing machine noisy →the model highligh retrieved chunk noisy spin cyle = bad bearings
So attention makes sure the model uses the right parts of context when answering
types of attention

Decoding

How the model choose the next word when generating the answer

Think of it like deciding what word comes next in a sentence

- Every LLM has a **default decoding method** (usually *Greedy* or *Top-p with temp=1.0*).
- But in RAG, **you should set it manually** → otherwise it may hallucinate.
    - Example: GPT defaults to a higher temperature (more creativity).
    - But in RAG QA, you want **low temperature (0.2–0.3)** for factual answers.

So → yes, **you choose decoding strategy** in code

In RAG → you normally stick with **Greedy** or **Top-p (with low temperature ~0.2–0.3)** for factual QA bots.

Techniques

1.Greedy decoding =always pick the single most likely word
Why noisy ?--> unbalanced load short ,strict

2.Top-k sampling =pick randomly from best k choices

If k=3 ->choices {unbalanced,bearings,motor}moght answers “bearings“

3.Top-p (nucleus sampling)=pick from smallest set of words whose probs >= p

If p =0.9 ->picks from words that together cover 90% probability

4.Temperature : control randomness

0.2 very factual answers

1.0 creative ,may invent things

5.Beam search =explores multiple possible answers and picks best

Might output :unbalanced load OR worn bearings

For rag chatbots →use greedy or top-p with low temp (0.2 -0.3) →ensures factual answers
For creative writing →higher temp,top-k

Post-processing

After answer is generated we clean & enforce rules

Detokenize → numbers back to text
Stop sequences →cut answer at a marker (##END)
Schema enforcement →makes sure answer follows json /markdown
Citation mapping → [1][2]link to correct chunks
Safety filters → removes personal info (PII-personally identifiable info)
Verification model →second LLm checks if answer is faithful

Raw answer

Your washing machine is noisy due to unbalanced loads bearings etc..

After post -processing

Your washing machine is noisy due to unbalanced loads or worn bearings [1][2]

Evaluation of RAG:

Manual evaluation :human check if answers ae correct and useful

Automatic evaluation :use metrics

Faithfulness is the answer grounded in retrireved docs?

Relevance is the chunk chosen related to the question

Precision /recall/F1 borrowed from search /ML precision =correctness,recall=completeness

Latency time to answer

Hallucination rate (answer not in docs)

Langsmith automatically using tool otherwise manual testing

Optimization

Make RAG faster and cheaper

Latency tricks (speed):

Batching(process multiple queries at once) ,streaming (send tokens a they are generated,not wait until the end)and prompt caching (reuse embedding for same/similar prompts)

Cost tricks:

Reduce chunk size or top-k (less data passed into LLM)
Compress context before feeding it
Use smaller models for reranking ,big models only for answering

Some of these are manual (deciding k.compression)some automatic (batching,streaming built into frameworks)

Advanced LLM integration

RAG isnt enough →you adapt the model

#Fine tuning (re train the LLm with your data)

Expensive big dataset needed

#Adapters /Lora/PEFt lightweight fine-tuning

Small add-on layers to adjust model
Cheaper,faster

#safety controls:

PII filtering(scrub personal identifiable information)
guardrails(rule to stop harmful or irrelevant answers)
Verification models (second smaller model checks if the answer is faithful)

Mostly manual setup (we have to design rules,filters)but some managed services #deployment

Local deployment run on our laptop server
Cloud deployment use aws,gcp,azure
Scaling methods horizontal more servers,vertical bigger servers and load balancing spread queries across servers
Monitoring & logging track latency,errors costs user queries
Detect hallucination unsafe outputs

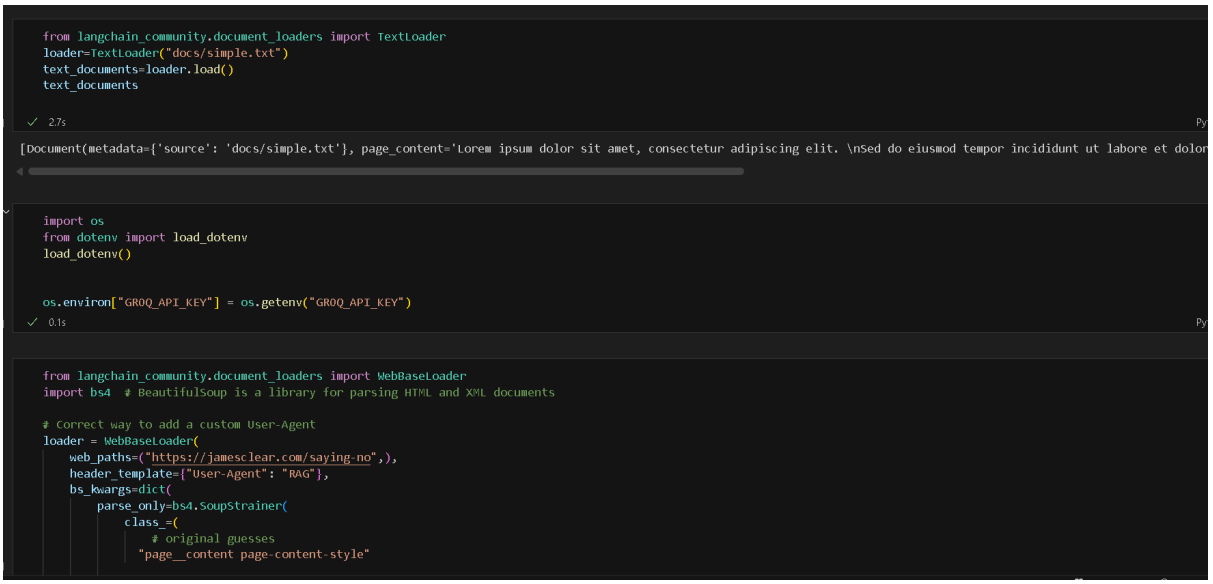
Evaluation → manual + automatic metrics (faithfulness, recall, hallucinations).

Optimization → tricks for speed & cost (batching, streaming, prompt caching).

Advanced integration → fine-tuning, adapters, guardrails, safety.

Deployment → local vs cloud, scaling, monitoring.

Link of the python file: [Tutorial1]



```
        )
    )
)

text_documents = loader.load()
print(text_documents)
```

✓ 2.1s Python

```
USER_AGENT environment variable not set, consider setting it to identify your requests.
[Document(metadata={'source': 'https://jamesclear.com/saying-no'}, page_content='\nThe ultimate productivity hack is saying no.\nNot doing something will always be faster than d
```

Python

```
from langchain_community.document_loaders import PyPDFLoader
loader=PyPDFLoader(r"C:\Users\subhalakshmi\Desktop\RAG\docs\RAG.pdf")
docs=loader.load()
docs
```

✓ 8.5s Python

```
[Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
```

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
documents=text_splitter.split_documents(docs)
documents[:10]
```

✓ 0.1s Python

```
[Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
 Document(metadata={'producer': 'Apache FOP Version 2.6', 'creator': 'ZonBook XSL Stylesheets with Apache FOP', 'creationdate': '2025-08-01T09:30:07+00:00', 'title': 'AWS Prescr
```

```
import os
from langchain_community.embeddings import CohereEmbeddings
from langchain_community.vectorstores import Chroma

load_dotenv()

# Get API key from env
api_key = os.getenv("COHERE_API_KEY")
if not api_key:
    raise ValueError("COHERE_API_KEY is not set in the environment.")

# Pass API key to embeddings explicitly
embeddings = CohereEmbeddings(cohere_api_key=api_key,user_agent="RAG")


# Create Chroma DB from first 20 documents
db = Chroma.from_documents(documents[:20], embeddings)
```

✓ 7.8s Python

```
C:\Users\subhalakshmi\AppData\Local\Temp\ipykernel_19552\3546758222.py:13: LangChainDeprecationWarning: The class `CohereEmbeddings` was deprecated in LangChain 0.0.30 and will I
  embeddings = CohereEmbeddings(cohere_api_key=api_key,user_agent="RAG")
```

```
query="what is RAG?"
result=db.similarity_search(query)
result[0].page_content
```

[7] ✓ 0.4s Python

```
... les an overview of building RAG systems on Amazon Web Services (AWS). By reviewing \nthe RAG options and architectures, you can choose between fully managed s
```

Generate + Code + Markdown

Start Chat to Generate Code (Ctrl+I)

Chatbot : [Chatbot]

```python
# 🔹 Step 2:

from langchain_community.document_loaders import UnstructuredFileLoader

# This loader automatically detects file type and extracts text
loader = UnstructuredFileLoader(r"C:\Users\subhalakshmi\Desktop\RAG\docs\LA Chatbot Info.docx")
docs = loader.load()

print(f" Loaded {len(docs)} document")
```

✓ 7.1s

```
C:\Users\subhalakshmi\AppData\Local\Temp\ipykernel_4316\1237727649.py:6: LangChainDeprecationWarning: The class `UnstructuredFileLoade
  loader = UnstructuredFileLoader(r"C:\Users\subhalakshmi\Desktop\RAG\docs\LA Chatbot Info.docx")  # replace with your file path
 Loaded 1 document
```

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Split docs into smaller chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n\n", "\n", " ", ""]
)

chunks = text_splitter.split_documents(docs)

print(f" Split into {len(chunks)} chunks")
```

[8] ✓ 0.0s

```
Split into 3 chunks
```

```python
from langchain_cohere import CohereEmbeddings

#  Initialize Cohere embeddings
embeddings = CohereEmbeddings(model="embed-english-v3.0")

#  Generate embeddings
all_embeddings = [embeddings.embed_query(chunk.page_content) for chunk in chunks]
all_embeddings
```

✓ 6.3s

```
[[-0.01158905,
  -0.055633545,
  -0.08068848,
  -0.051086426,
  -0.028961182,
  -0.011550903,
  -0.032287598,
  0.054107666,
  -0.0155181885,
  0.026947021,
  -0.01272583,
  -0.0032863617,
  -0.014831543,
```

```python
import os, chromadb
from dotenv import load_dotenv

#  1. Load environment variables
load_dotenv()

#  2. Connect to Chroma Cloud with API credentials
chroma = chromadb.CloudClient(
    api_key=os.environ["CHROMA_API_KEY"],
    tenant=os.environ["CHROMA_TENANT"],
    database=os.environ["CHROMA_DATABASE"],
)

collection = chroma.get_or_create_collection("least_action")
texts = [chunk.page_content for chunk in chunks]
metadatas = [chunk.metadata for chunk in chunks]
ids = [f"chunk-{i}" for i in range(len(texts))]

# 3. Store into Chroma
collection.upsert(
    ids=ids,
    documents=texts,
    embeddings=all_embeddings,
    metadatas=metadatas
)

print(collection.peek())
#res = collection.get(include=["embeddings", "documents", "metadatas"])
#print(res)
```

6.0s

```
ds': ['chunk-0', 'chunk-1', 'chunk-2'], 'embeddings': array([[-0.01158905, -0.05563354, -0.08068848, ...,  0.10778809,
       0.01248169,  0.03527832],
```

```python
from langchain.prompts import PromptTemplate

# === System Prompt Template ===
system_prompt = """
Hello! I'm your LA assistant. How can I help you today?

Guidelines for answering:
1. Always use the provided CONTEXT to answer the user's question.
2. If the answer is not in the CONTEXT and the question is irrelevant or unusual, politely say:
   "Apologies for the inconvenience, I don't know the answer to that."
3. If the question is about the company, its services, or in-depth details
   but the answer is not in the CONTEXT, politely say:
   "For further details, please contactus@leastactioncompany.com | +91-8825965775 | +91-9597366741"
4. End every response with a polite closing:
   "Thank you for connecting with me!"

---
CONTEXT:
{context}

QUESTION:
{question}

Answer:
"""

# Build the prompt template
prompt = PromptTemplate(
    input_variables=["context", "question"],
    template=system_prompt
)
```

```python
from langchain_groq import ChatGroq
from langchain.chains import StuffDocumentsChain
from langchain.chains.combine_documents import create_stuff_documents_chain
from dotenv import load_dotenv
import os


load_dotenv()
GROQ_API_KEY = os.getenv("GROQ_API_KEY")

llm = ChatGroq(
    groq_api_key=GROQ_API_KEY,
    model="llama-3.1-8b-instant"
)
qa_chain = create_stuff_documents_chain(
    llm=llm,
    prompt=prompt,
    document_variable_name="context"

)

def chat_with_bot(user_question):
    docs = retrieve(user_question, top_k=3)
    if isinstance(docs[0], list):
        docs = [doc for sublist in docs for doc in sublist]

    return qa_chain.invoke({
        "context": docs,
        "question": user_question
    })
```

```python
def chat_with_bot(user_question):
    docs = retrieve(user_question, top_k=3)
    if isinstance(docs[0], list):
        docs = [doc for sublist in docs for doc in sublist]

    return qa_chain.invoke({
        "context": docs,
        "question": user_question
    })

if __name__ == "__main__":
    while True:
        query = input("You: ")
        if query.lower() in ["exit", "quit"]:
            break
        answer = chat_with_bot(query)
        print("Bot:", answer)
```

13m 54.3s

```
Bot: Hello! It's great to connect with you. I'm your LA assistant. How can I help you today?

Thank you for connecting with me!
Bot: We provide a variety of services that cater to different needs. Our services include:
```

13m 54.3s

Bot: Hello! It's great to connect with you. I'm your LA assistant. How can I help you today?

Thank you for connecting with me!
Bot: We provide a variety of services that cater to different needs. Our services include:

1. Web Development: We design and develop custom websites that meet your business requirements.
2. Mobile App Development: We create engaging mobile apps for both Android and iOS platforms.
3. AI & ML Solutions: We offer artificial intelligence and machine learning solutions to help businesses automate processes and make data-driven decisions.
4. Digital Marketing: We provide digital marketing services, including SEO, social media marketing, and content marketing.
5. HRMS/ERP Software: We design and develop custom HRMS and ERP software to manage your business operations.
6. Custom software Solutions: We develop custom software solutions tailored to your business needs.
7. Cloud Integration: We help businesses integrate their applications with cloud services.
8. UI/UX Design: We design user-friendly and visually appealing interfaces for your web and mobile applications.
9. Careers: We offer internships and job openings for MERN, AI/ML, and BDE roles.
10. Client & Project Support: We provide support services, including project management, communication, and technical support.

Thank you for connecting with me!
Bot: Hello! I'm your LA assistant. How can I help you today?